Context

Target is one of the world's most recognized brands and one of America's leading retailers. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This business case has information of 100k orders from 2016 to 2018 made at Target in Brazil. Its features allow viewing an order from multiple dimensions: from order status, price, payment and freight performance to customer location, product attributes and finally reviews written by customers.

Dataset: https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb

Data is available in 8 csv files:

1. customers.csv

2. geolocation.csv

3. order_items.csv

4. payments.csv

5. reviews.csv

6. orders.csv

7. products.csv

8. sellers.csv

Each feature or columns of different CSV files are described below:

The **customers.csv** contain following features:

Features	**Description**
customer_id	Id of the consumer who made the purchase.
customer_unique_id	Unique Id of the consumer.
customer_zip_code_prefix	Zip Code of the location of the consumer.
customer_city	Name of the City from where order is made.
customer_state	State Code from where order is made(Ex- sao paulo-SP).

The **sellers.csv** contains following features:

Features	**Description**
seller_id	Unique Id of the seller registered
seller_zip_code_prefix	Zip Code of the location of the seller.
seller_city	Name of the City of the seller.
seller_state	State Code (Ex- sao paulo-SP)

The **order_items.csv** contain following features:

Features	**Description**
order_id	A unique id of order made by the consumers.
order_item_id	A Unique id given to each item ordered in the order.
product_id	A unique id given to each product available on the site.
seller_id	Unique Id of the seller registered in Target.
shipping_limit_date	The date before which shipping of the ordered product must be completed.
price	Actual price of the products ordered.
freight_value	Price rate at which a product is delivered from one point to another.

The **geolocations.csv** contain following features:

Features	Description
geolocation_zip_code_prefix	first 5 digits of zip code
geolocation_lat	latitude
geolocation_lng	longitude
geolocation_city	city name
geolocation_state	state

The **payments.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
payment_sequential	sequences of the payments made in case of EMI.
payment_type	mode of payment used. (Ex-Credit Card)
payment_installments	number of instalments in case of EMI purchase.
payment_value	Total amount paid for the purchase order.

The **orders.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
customer_id	Id of the consumer who made the purchase.
order_status	status of the order made i.e delivered, shipped etc.
order_purchase_timestamp	Timestamp of the purchase.
order_delivered_carrier_date	delivery date at which carrier made the delivery.
order_delivered_customer_date	date at which customer got the product.
order_estimated_delivery_date	estimated delivery date of the products.

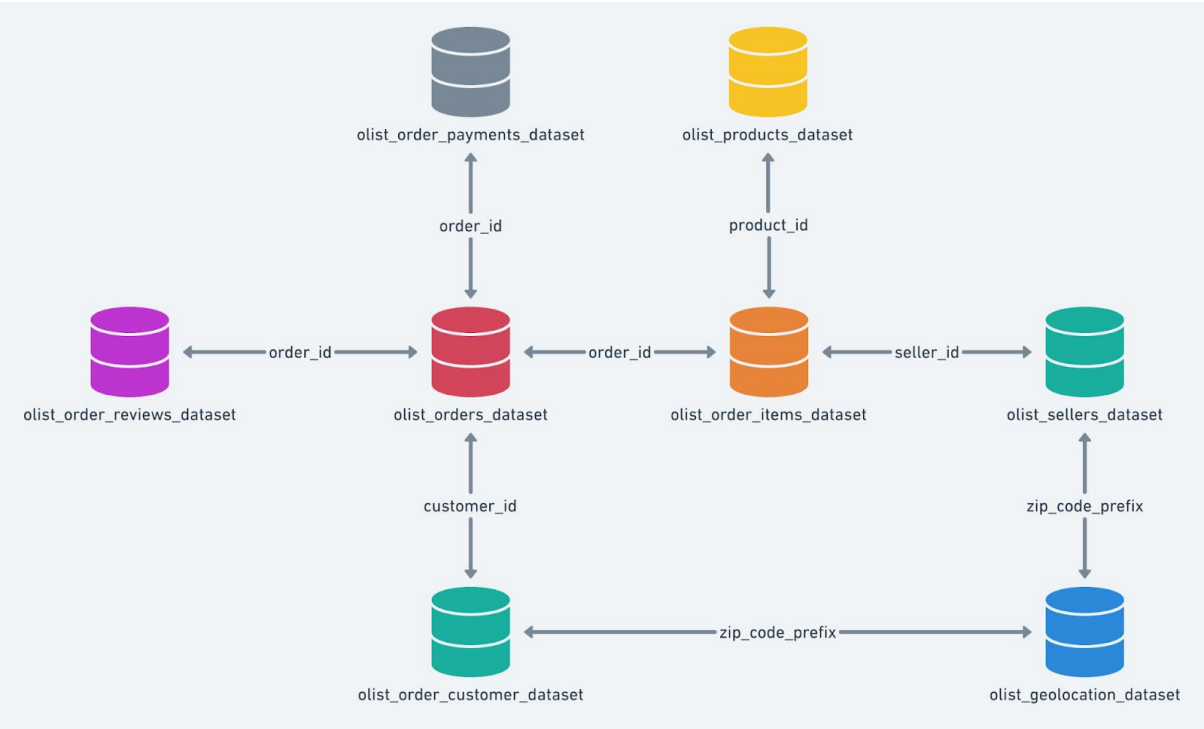
The **reviews.csv** contain following features:

Features	Description
review_id	Id of the review given on the product ordered by the order id.
order_id	A unique id of order made by the consumers.
review_score	review score given by the customer for each order on the scale of 1–5.
review_comment_title	Title of the review
review_comment_message	Review comments posted by the consumer for each order.
review_creation_date	Timestamp of the review when it is created.
review_answer_timestamp	Timestamp of the review answered.

The **products.csv** contain following features:

Features	Description
product_id	A unique identifier for the proposed project.
product_category_name	Name of the product category
product_name_lenght	length of the string which specifies the name given to the products ordered.
product_description_lenght	length of the description written for each product ordered on the site.
product_photos_qty	Number of photos of each product ordered available on the shopping portal.
product_weight_g	Weight of the products ordered in grams.
product_length_cm	Length of the products ordered in centimetres.
product_height_cm	Height of the products ordered in centimetres.
product_width_cm	width of the product ordered in centimetres.

High level overview of relationship between datasets:



Assume you are a data scientist at Target, and are given this data to analyze and provide some insights and recommendations from it.

What does 'good' look like?

1. **Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset.**

Data type of columns in a table

Query:

```
select column_name, data_type
from `case-study-383004.case_study.INFORMATION_SCHEMA.COLUMNS`
where table_name = 'customers'
order by ordinal_position
```

Query Result Table: -

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

Observation:-

- In this data some columns are having the string data and some integer data also.
- But Customer_id having the integers and characters both so in big query we consider it as “String” only, generally it is a “Varchar”.
- Zip code must be integers only so that should be integer data type.

Recommendations: -

- To know about the data types of data use this above syntax.

Time period for which the data is given.

Query:

```
SELECT order_purchase_timestamp, order_purchase_timestamp, order_purchase_timestamp,
        order_purchase_timestamp
FROM `case-study-383004.case_study.orders` LIMIT 10

SELECT review_answer_timestamp, review_answer_timestamp, review_answer_timestamp,
        review_answer_timestamp
FROM `case-study-383004.case_study.order_reviews` LIMIT 10
```

Query Result Table: -

Query results SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	order_purchase_timestamp	order_purchase_timestamp_1	order_purchase_timestamp_2	order_purchase_timestamp_3
1	2017-11-25 11:10:33 UTC	2017-11-25 11:10:33 UTC	2017-11-25 11:10:33 UTC	2017-11-25 11:10:33 UTC
2	2017-12-05 01:07:58 UTC	2017-12-05 01:07:58 UTC	2017-12-05 01:07:58 UTC	2017-12-05 01:07:58 UTC
3	2017-12-05 01:07:52 UTC	2017-12-05 01:07:52 UTC	2017-12-05 01:07:52 UTC	2017-12-05 01:07:52 UTC
4	2018-02-09 17:21:04 UTC	2018-02-09 17:21:04 UTC	2018-02-09 17:21:04 UTC	2018-02-09 17:21:04 UTC
5	2017-11-06 13:12:34 UTC	2017-11-06 13:12:34 UTC	2017-11-06 13:12:34 UTC	2017-11-06 13:12:34 UTC
6	2017-04-20 12:45:34 UTC	2017-04-20 12:45:34 UTC	2017-04-20 12:45:34 UTC	2017-04-20 12:45:34 UTC
7	2017-07-13 11:03:05 UTC	2017-07-13 11:03:05 UTC	2017-07-13 11:03:05 UTC	2017-07-13 11:03:05 UTC
8	2017-07-11 13:36:30 UTC	2017-07-11 13:36:30 UTC	2017-07-11 13:36:30 UTC	2017-07-11 13:36:30 UTC
9	2017-07-29 18:05:07 UTC	2017-07-29 18:05:07 UTC	2017-07-29 18:05:07 UTC	2017-07-29 18:05:07 UTC
10	2017-07-13 10:02:47 UTC	2017-07-13 10:02:47 UTC	2017-07-13 10:02:47 UTC	2017-07-13 10:02:47 UTC

Observation: -

- In this orders and order items both have the time periods.
- Both are giving the time stamp between orders and purchasing periods and estimated time also.

Recommendations: -

- Above queries both are representing the orders when we give.
- If want to particular days we can use different functions and create days, months and years separately.

Cities and States of customers ordered during the given period.

Query:

```
select c.customer_city, c.customer_state, Count(o.order_approved_at) AS order_state,
extract(year from o.order_purchase_timestamp) AS period
from `case_study.customers` AS c
join `case_study.orders` AS o
on c.customer_id = o.customer_id
Group By c.customer_city,c.customer_state, period
LIMIT 10
```

Query Result Table: -

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_city	customer_state	order_state	period
1	rio de janeiro	RJ	3336	2017
2	sao leopoldo	RS	52	2017
3	general salgado	SP	1	2017
4	brasilia	DF	1211	2018
5	paranavai	PR	19	2017
6	cuiaba	MT	98	2017
7	sao luis	MA	169	2017
8	maceio	AL	106	2017
9	hortolandia	SP	68	2017
10	varzea grande	MT	18	2017

Observation: -

- In this above case there is two tables involved. Those are customers and orders tables.
- Customers tables are having cities and states, orders tables having periods of orders.
- So here I observed during the period of years so many customers are orders during the periods.

Recommendations: -

- During the period how many customers involved from which state and cities we use both customers and orders tables.

2. **In-depth Exploration:**

Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?

Query:

```
SELECT Extract(year FROM order_purchase_timestamp) AS Year,
    Extract(month FROM order_purchase_timestamp) AS Month,
    COUNT(1) AS Num_Orders
FROM `case-study-383004.case_study.orders`
Group By Year, Month
Order By Year, Month
```

Query Result Table: -

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	Year	Month	Num_Orders
1	2016	9	4
2	2016	10	324
3	2016	12	1
4	2017	1	800
5	2017	2	1780
6	2017	3	2682
7	2017	4	2404
8	2017	5	3700
9	2017	6	3245
10	2017	7	4026

Observation: -

- In this query I observed the seasonally growing the orders.
- Here some of months have the more orders and some moths have less orders.

Recommendations: -

- In this case we easily find out the seasonal orders and looking this we easily take the decision to improve our business orders.

What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?

3. **Evolution of E-commerce orders in the Brazil region:**

Get month on month orders by states

Query:

```
select extract(month from o.order_purchase_timestamp) AS mnth, c.customer_state, count(o.
order_id)AS num_orders
from `case_study.customers` AS c
JOIN `case_study.orders` AS o
ON c.customer_id = o.customer_id
group by 1,2
order by 1
```

Query Result Table: -

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	mnth	customer_state	num_orders
1	1	RJ	990
2	1	SP	3351
3	1	DF	151
4	1	RS	427
5	1	CE	99
6	1	PE	113
7	1	PR	443
8	1	BA	264
9	1	MG	971
10	1	RN	51

Observation: -

- Here every month-on-month sales is displaying.
- So, some moths number of orders is more and some months number of orders is less.
- Above query we grouped by months wise and states wise

Recommendations: -

- In query we are finding month on month orders from different states.
- So, to see this table which states are having less orders we need to increase the orders easily.

Distribution of customers across the states in Brazil

Query:

```
select customer_state, count(customer_id) AS no_cust
from `case_study.customers`
group by 1
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	no_cust
1	RN	485
2	CE	1336
3	RS	5466
4	SC	3637
5	SP	41746
6	MG	11635
7	BA	3380
8	RJ	12852
9	GO	2020
10	MA	747
11	PE	1652

Observation: -

- In this above query shows the distribution of customers as per states.
- So, here some states should be less customers and some more customers.

Recommendations: -

- To see this table, we easily find out the customers according to states wise.

4. **Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight, and others.**

Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use "payment_value" column in payments table.

Query:

```
with a as (
select extract(year from o.order_purchase_timestamp) AS yr, sum(p.payment_value) AS cost
_orders
from `case_study.orders` AS o
JOIN `case_study.payments` AS p
ON o.order_id = p.order_id
where extract(month from o.order_purchase_timestamp) between 1 and 8
Group by 1)

select round(((a1.cost_orders/a2.cost_orders)-1)*100,2) AS perc_increase
from a as a1, a as a2
where a1.yr = 2018 and a2.yr = 2017
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	perc_increase
1	136.98

Observation: -

- In this above query shows percentage increase in cost of orders from 2017 to 2018.
- So, the result is 136.98% increase from 2017 to 2018.

Recommendations: -

- To see this I see the orders and payments table orders increase year by year.

.

Mean & Sum of price and freight value by customer state.

Query:

```
select c.customer_state,avg(oi.price) as Mean_price,
    sum(oi.price) as Sum_price,
    AVG(oi.freight_value) AS AVG_freight_value,
    Sum(oi.freight_value) AS Total_freight
FROM `case_study.customers` AS c
JOIN `case_study.orders` AS o
ON c.customer_id = o.customer_id
JOIN `case_study.order_items` AS oi
on o.order_id = oi.order_id
group by 1
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	Mean_price	Sum_price	AVG_freight_valu	Total_freight
1	MT	148.297184...	156453.529...	28.1662843...	29715.4300...
2	MA	145.204150...	119648.219...	38.2570024...	31523.7700...
3	AL	180.889211...	80314.81	35.8436711...	15914.5899...
4	SP	109.653629...	5202955.05...	15.1472753...	718723.069...
5	MG	120.748574...	1585308.02...	20.6301668...	270853.460...
6	PE	145.508322...	262788.029...	32.9178626...	59449.6599...
7	RJ	125.117818...	1824092.66...	20.9609239...	305589.310...
8	DF	125.770548...	302603.939...	21.0413549...	50625.4999...
9	RS	120.337453...	750304.020...	21.7358043...	135522.740...
10	SE	153.041168...	58920.8500...	36.6531688...	14111.4699...

Observation: -

- Above table shows the Mean, sum, freight and total freight charges during the customer states.
- It is different from total charges of the states how much.

Recommendations: -

- To see the changes between the states we use this query and find out the mean and freight charges.

5. **Analysis on sales, freight, and delivery time**

Calculate days between purchasing, delivering and estimated delivery.

Query:

```
select
    date_diff(order_purchase_timestamp, order_delivered_customer_date, Day) AS Diff_Pur_Del,
    date_diff(order_estimated_delivery_date, order_delivered_customer_date, Day) AS Diff_Est_Del
from `case_study.orders`
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	Diff_Pur_Del	Diff_Est_Del
1	-30	-12
2	-30	28
3	-35	16
4	-30	1
5	-32	0
6	-29	1
7	-43	-4
8	-40	-4
9	-37	-1
10	-33	-5

Observation: -

- Above table shows the days between purchasing, delivering and estimated delivery.
- Observe this table purchasing to delivery so much of days is there and estimated to delivery also so much of days is there.

Recommendations: -

- If we take the different of the estimated and delivery and purchase to delivery there is a so much of time difference is there.

Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:

- time_to_delivery = order_purchase_timestamp-order - delivered_customer_date
- diff_estimated_delivery=order_estimated_delivery_date-
  order_delivered_customer_date

Query:

```
SELECT date_diff(order_purchase_timestamp, order_delivered_customer_date, Day) AS Ti
me_to_delivery,
    date_diff(order_estimated_delivery_date, order_delivered_customer_date, Day) AS diff_e
stimated_delivery
from `case_study.orders`
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	Time_to_delivery	diff_estimated_c
1	-30	-12
2	-30	28
3	-35	16
4	-30	1
5	-32	0
6	-29	1
7	-43	-4
8	-40	-4
9	-37	-1
10	-33	-5
11	-38	-6

Observation: -

- Above table shows the days between purchasing, delivering and estimated delivery.
- Observe this table purchasing to delivery so much of days is there and estimated to delivery also so much of days is there.

Recommendations: -

- If we take the different of the estimated and delivery and purchase to delivery, there is a so much of time difference is there.

Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery.

Query:

```
SELECT customer_state,AVG(freight_value) AS Mean_Freight_Value,
      date_diff(order_purchase_timestamp, order_delivered_customer_date, Day) AS Time_to
_delivery,
     date_diff(order_estimated_delivery_date, order_delivered_customer_date, Day) AS diff_e
stimated_delivery

from `case_study.orders`
JOIN `case_study.order_items`
ON `case_study.orders`.order_id =`case_study.order_items`.order_id
JOIN `case_study.customers`
ON `case_study.orders`.customer_id = `case_study.customers`.customer_id
GROUP BY `case_study.customers`.customer_state,Time_to_delivery, diff_estimated_delive
ry
ORDER BY customer_state DESC
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	Mean_Freight_V	Time_to_deliver	diff_estimated_c
1	TO	25.564	null	null
2	TO	17.9433333...	-14	12
3	TO	17.78	-24	8
4	TO	39.51	-30	-8
5	TO	37.1	-19	0
6	TO	42.1866666...	-20	13
7	TO	53.88	-22	-2
8	TO	40.21	-10	14
9	TO	90.62	-14	14
10	TO	37.0100000...	-22	6

Observation: -

- Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery.
- There should be mean value and time to delivery and difference between the estimated date and delivery date.
- Here we observe the value to one out of the other dates.

Recommendations: -

- If we take the data between those means, we recommend that use this query.

Sort the data to get the following:

Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5.

Query:

```
SELECT `case_study.customers`.customer_state,
    AVG(freight_value) AS Mean_Freight_Value
from `case_study.orders`
JOIN `case_study.order_items`
ON `case_study.orders`.order_id =`case_study.order_items`.order_id
JOIN `case_study.customers`
ON `case_study.orders`.customer_id = `case_study.customers`.customer_id
GROUP BY `case_study.customers`.customer_state
ORDER BY Mean_Freight_Value DESC
LIMIT 5
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	Mean_Freight_V
1	RR	42.9844230...
2	PB	42.7238039...
3	RO	41.0697122...
4	AC	40.0733695...
5	PI	39.1479704...

Observation: -

- Above table shows the top five freight charges is showing.
- Because the freight charges are different than the different states.

Recommendations: -

- To know the mean of freight charges use this query.

Top 5 states with highest/lowest average time to delivery

Query:

```
SELECT c.customer_state,
    AVG(date_diff(order_purchase_timestamp, order_delivered_customer_date, Day)) AS A
VG_Time_to_delivery
from `case_study.orders` AS o
JOIN `case_study.customers` AS c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state
ORDER BY AVG_Time_to_delivery, customer_state DESC
LIMIT 5
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	customer_state	AVG_Time_to_de
1	RR	-28.9756097...
2	AP	-26.7313432...
3	AM	-25.9862068...
4	AL	-24.0403022...
5	PA	-23.3160676...

Observation: -

- In this table showing the average of delivery to time to estimate to each state.
- Those all should be negative because there should be estimated to delivery date first, so we get the negative balance.
- In case estimated date is first and delivery date is to be next we get the positive days.

Recommendations: -

- In the I see, if we show the mean of the delivery date to use this query.

Top 5 states where delivery is really fast/ not so fast compared to estimated date.

Query:

```
SELECT c.customer_state,
    date_diff(order_delivered_customer_date, order_estimated_delivery_date, Day) AS Fast_
delivery
from `case_study.orders` AS o
JOIN `case_study.customers`AS c
ON o.customer_id = c.customer_id
GROUP BY c.customer_state,Fast_delivery
ORDER BY Fast_delivery DESC
LIMIT 5
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	customer_state	Fast_delivery
1	RJ	188
2	ES	181
3	SP	175
4	SP	167
5	SE	166

Observation: -

- In this query to show the fast delivery to estimated.
- So, in this more days is there meaning the product is delivered fast.
- If it is negative days, there means the product is lately delivered.

Recommendations: -

- To know the item delivered fast or not to use this query.

6. **Payment type analysis:**

Month over Month count of orders for different payment types.

Query:

```
SELECT extract(month from o.order_purchase_timestamp) AS mnth,
    p.payment_type, count(distinct o.order_id) AS num_orders
FROM `case-study-383004.case_study.orders` AS o Join `case-study-
383004.case_study.payments` AS p
ON o.order_id = p.order_id
GROUP BY 1,2
order by 1;
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	mnth	payment_type	num_orders
1	1	credit_card	6093
2	1	UPI	1715
3	1	voucher	337
4	1	debit_card	118
5	2	UPI	1723
6	2	credit_card	6582
7	2	voucher	288
8	2	debit_card	82
9	3	credit_card	7682
10	3	UPI	1942

Observation: -

- The above table shows the different payment methods.
- Here so many customers using the credit card option.
- We get to know the which is more available to customers to paid.

Recommendations: -

- To use this query we strongly recommend that next future may be UPI payments must be increase, in case if we give any rewards.

Count of orders based on the no. of payment instalments.

Query:

```
select payment_installments, count(distinct order_id) AS num_orders
From `case-study-383004.case_study.payments`
group by 1;
```

Query Result Table:

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	payment_installr	num_orders
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644
11	10	5315

Observation: -

- The above table shows the on which instalment how many orders got.
- To see this, we got to know the orders per instalments.

Recommendations: -

- To above this instalment should be the depends on the orders.